



Zadania praktyczne — Szkolenie UiPath — Dzień 4

Autoryzowany Partner UiPath

info@robotivity.pl · www.Robotivity.pl · tel.: +48 690 020 576

DZIEŃ 4

MODUŁ 1 — STATE MACHINE: WPROWADZENIE

Ćwiczenie 1 — State Machine: Input Dialog z rozgałęzieniem

Folder: Dzień4_Zadanie1_StateMachine_Wprowadzenie/ · Temat: State Machine + Transitions + pętla

OPIS ZADANIA

- 1. Użytkownik wpisuje imię przez `Input Dialog`
- 2. Jeśli **"Adam"** → przypisz nazwisko "Nowak"
- 3. Jeśli **"Ewa"** → przypisz nazwisko "Kowalska"
- 4. Inne imię → ponów pytanie (pętla!) z Log Message: "Podane imie nie jest Adam ani Ewa."
- 5. Wyświetl pełne imię i nazwisko w Log Message

STANY

Stan	Entry	Exit	Transitions
State - Init (Initial)	<code>Input Dialog</code> → <code>str_imie2</code>	Log: "Przechodze przez Exit w State Init"	<code>str_imie2 = "Adam"</code> → State Adam; <code>str_imie2 = "Ewa"</code> → State Ewa; inne (Action: Log "Podane imie nie jest Adam ani Ewa.") → State Init (pętla)
State - Adam	<code>str_imie2 = str_imie2 & "Nowak"</code>	Log: "Przechodze przez Exit - Adam"	(bezwarunkowa) → Final State
State - Ewa	<code>str_imie2 = str_imie2 & "Kowalska"</code>	Log: "Przechodze przez Exit - Ewa"	(bezwarunkowa) → Final State
Final State	Log Message: "Twoja osoba to: " & <code>str_imie2</code>	—	— (koniec)

Wskazówka: Scope zmiennych — `str_imie2` musi być na poziomie StateMachine (nie wewnątrz State). Kolejność Transitions jest ważna — pierwsza spełniona wygrywa. Każdy State ma **Entry** i **Exit**. Transition ma opcjonalne pole **Action** (np. Log Message).

ZMIENNE

Nazwa	Typ	Scope	Opis
<code>str_imie2</code>	String	StateMachine	Imię wpisane przez użytkownika

MODUŁ 2 — DICTIONARY

Ćwiczenie 2 — Kursy walut NBP → Dictionary

Folder: `Dzien4_Zadanie2_Dictionary_Waluty/` · Temat: `Table Extraction + Dictionary + dostęp po kluczu`

OPIS ZADANIA

- Otwórz stronę NBP z tabelą kursów walut (tabela A)
- Pobierz tabelę kursów (`Extract Table Data`)
- Zapisz tabelę do `Waluty.xlsx`
- Zainicjalizuj Dictionary: `dict_waluty = New Dictionary(Of String, String)`
- W pętli `For Each Row` : klucz = kod waluty, wartość = kurs
- Log Message: `"Kurs euro to: " & dict_waluty("1 EUR")`

Uwaga: Dictionary **MUSI** być zainicjalizowane w Assign: `New Dictionary(Of String, String)` — inaczej `NullReferenceException` !

ZMIENNE

Nazwa	Typ	Opis
<code>dict_waluty</code>	<code>Dictionary(Of String, String)</code>	Słownik walut: kod → kurs
<code>dt_waluty</code>	<code>DataTable</code>	Dane z tabeli kursów NBP
<code>str_key_kod_waluty</code>	<code>String</code>	Klucz (np. "1 EUR")
<code>str_value_wartosc</code>	<code>String</code>	Wartość kursu

BONUS Zadanie dodatkowe — Tłumacz walut (interaktywny)

Temat: `While + Input Dialog + Dictionary.ContainsKey + walidacja`

OPIS ZADANIA

- Podczas ładowania Dictionary (kroki 1-5 z Ćwiczenia 2), dodaj obliczenie jednostkowej wartości każdej waluty oraz usuń liczby z kodu waluty
- Pętla `While` z `bl_kontynuuj` :
 - `Input Dialog` — "Podaj kod waluty (np. 'EUR') lub 'koniec'"
 - Jeśli "koniec" → zakończ pętlę
 - Jeśli `dict_waluty.ContainsKey(str_kod)` → Message Box z kursem
 - Jeśli nie → Message Box: "Nie znaleziono waluty"

MODUŁ 3 — ROZBUDOWANE STATE MACHINE

Ćwiczenie 3 — Rozbudowane State Machine z obsługą błędów

Folder: *Dzien4_Zadanie3_Rozbudowane_StateMachine/* · Temat: *State Machine + BusinessException + SystemException + Robotivity*

OPIS ZADANIA

Stwórz State Machine z 4 stanami przetwarzającą transakcje z Excela w Robotivity:

STANY PROCESU

Stan	Opis	Transitions
State - Init	Config → Dictionary, Kill Process, otwórz Robotivity, zaloguj (Retry Scope)	Sukces → Pobierz trans.; SE → Final State
State - Pobierz transakcję	Sprawdź TransactionNumber, pobierz DataRow	Sukces → Procesowanie; Koniec → Final State
State - Procesowanie	Wpisz dane do Robotivity, obsłuż BE/SE	Sukces → Pobierz trans.; SE → Init (restart!)
Final State	Log status, Kill Process	— (koniec)

KLUCZOWE ELEMENTY

- **BusinessRuleException** — dane nienumeryczne → pomiń transakcję (NIE ponawiaj)
- **SystemException** — błąd techniczny → restart Init (PONÓW)
- Czyszczenie `BusinessRuleException = Nothing` na początku procesowania
- Dictionary jako Config — wzorzec REFramework
- Retry Scope wokół logowania — 2 próby, 3s odstęp

ZMIENNE

Nazwa	Typ	Opis
int_transaction_number	Int32 (default=1)	Numer bieżącego wiersza
dict_Config	Dictionary(Of String, String)	Ustawienia
dt_transakcje	DataTable	Dane z Transactions.xlsx
row_TransactionItem	DataRow	Bieżący wiersz
SystemException	Exception	Wyjątek systemowy
BusinessRuleException	BusinessRuleException	Wyjątek biznesowy

Ćwiczenie 4 — REFramework: Wprowadzenie (demo)

Folder: `Dzien4_Zadanie4_RoboticEnterpriseFramework_Wprowadzenie/` · Temat: Struktura REFramework + `Config.xlsx` + `Main.xaml`

OPIS ĆWICZENIA (Z PROWADZĄCYM)

1. `File` → `New` → `Process` → szablon **"Robotic Enterprise Framework"**
2. Przejrzyj strukturę: `Main.xaml` , `Framework/` , `Data/Config.xlsx`
3. Uruchom "out of the box" (F5) — obserwuj logi
4. Edytuj `Config.xlsx` — zmień `OrchestratorQueueName`
5. Otwórz `Process.xaml` — tu wpisujesz logikę biznesową

CONFIG.XLSX — TRZY ARKUSZE

Arkusz	Opis	Przykład
Settings	Ustawienia procesu	OrchestratorQueueName, MaxRetryNumber
Constants	Stałe wartości	Timeout_General = 30000
Assets	Nazwy assetów z Orchestratora	CredentialName = "Robot_Login"

Uwaga: W arkuszu Assets nie wpisujemy haseł — tylko **nazwy assetów** z Orchestratora.

Ćwiczenie 5a — Dispatcher

Folder: `Dzien4_Zadanie5a_Dispatcher/` · Temat: Try Catch + Add Queue Items do kolejki Orchestratora

OPIS ZADANIA

- 1. Try Catch wokół Read Range :
 - Try: Read Range Workbook — wczytaj Transactions.xlsx (AddHeaders = True)
 - Catch (Exception): Kill Process "EXCEL" → Delay 2s → ponów Read Range
- 2. (Opcja A — Bulk) Bulk Add Queue Items :
 - QueueName: Robotivity_Bulk
 - FolderPath: Robotivity2
- 3. (Opcja B — po kolei z referencją) For Each Row → Add Queue Item :
 - QueueType: Rorbitivity_Single
 - FolderPath: Robotivity2
 - Reference: CashIn & "_" & OnUsCheck
 - ItemInformation: CashIn , OnUsCheck , NotOnUsCheck
- 4. Log Message: "Transakcje dodane do kolejki"

Wskazówka: Try Catch wokół Read Range zabezpiecza przed zablokowanym plikiem Excel. Opcja Bulk dodaje wszystko na raz, opcja z pętlą pozwala nadać unikatowe referencje.

ZMIENNE

Nazwa	Typ	Opis
dt_transakcje	DataTable	Dane z Transactions.xlsx

Ćwiczenie 5b — REFramework Performer

Folder: `Dzien4_Zadanie5b_RoboticEnterpriseFramework_Performer/` · Temat: `REFramework + Queue + Robotivity + BusinessException`

CO NALEŻY SKONFIGUROWAĆ

1. `Config.xlsx`:

- Settings: `OrchestratorQueueName` = `4a_Dispatcher`
- Settings: `OrchestratorQueueFolder` = `Nazwa Folderu w Orchestrator`
- Assets: nazwy assetów (credentials, ścieżka)

2. `InitAllApplications.xaml`:

- Get Credential → Username, Password
- Get Asset → ścieżka do Robotivity
- Otwórz Robotivity i zaloguj się

3. `Process.xaml`:

- Wyciągnij dane z `in_TransactionItem.SpecificContent`
- Sprawdź `.IsNumeric`
- Numeryczne → Type Into + Click Accept
- Nienumeryczne → `Throw New BusinessException("Dane nie sa numeryczne")`

4. `CloseAllApplications.xaml`: `Kill Process "robotivity"`

Wskazówka: REFramework automatycznie obsługuje retry, logging, status transakcji i restart po System Exception. Programista edytuje głównie: `Process.xaml`, `InitAllApplications.xaml`, `Config.xlsx`.

BONUS Zadanie bonusowe — Logowanie wyników do Excela

Temat: `Modyfikacja SetTransactionStatus.xaml + Append Range`

OPIS ZADANIA

- W `SetTransactionStatus.xaml` po ustawieniu statusu:
- Oblicz status i referencję transakcji
- `Append Range` → dopisz wiersz do `Data/Output/TransactionLog.xlsx`

BONUS Zadanie bonusowe — Wysyłka emaila z raportem

Temat: `Podsumowanie po zakończeniu REFramework + SMTP`

OPIS ZADANIA

- W `CloseAllApplications.xaml` (przed Kill Process):
- Policz transakcje Successful i Failed
- Stwórz treść emaila z podsumowaniem
- Wyślij przez SMTP

